LESSON 3

Hash Maps (Dictionaries)

~40-50% of easy-medium problems solved with this one tool.

3.1 What is a Hash Map & Why It's Fast

```
Think of it as a MAGIC NOTEBOOK:
  - Write "apple -> 3" (store)
  - Ask "what's apple?" (lookup) -> 3 INSTANTLY

List:     "Is 7 in [3,1,4,1,5,9,2,6]?" -> check ALL elements -> O(n)
Hash Map: "Is 7 in {3:T,1:T,4:T,...}?"  -> instant answer   -> O(1)

WHY O(1)? Hash map computes a hash of the key, which gives
the exact memory location. No searching needed.

In Python: dict IS a hash map
           set IS a hash set (keys only, no values)
```

3.2 Counting Pattern

The most common hash map pattern. Loop through data, count occurrences.

```
# PATTERN (memorize this skeleton):
count = {}
for item in data:
    count[item] = count.get(item, 0) + 1
#                       ^^^^^^^^^^
#                 "if missing, use 0 as default"

# Example: Count character frequency
def count_chars(s):
    count = {}
    for char in s:
        count[char] = count.get(char, 0) + 1
    return count

count_chars("banana")  # {'b':1, 'a':3, 'n':2}

# SHORTCUT: use Counter from collections
from collections import Counter
Counter("banana")      # Counter({'a':3, 'n':2, 'b':1})
Counter("banana").most_common(2)  # [('a',3), ('n',2)]
```

```
count_chars("banana")

char='b': count.get('b',0)=0, count={'b':1}
char='a': count.get('a',0)=0, count={'b':1,'a':1}
char='n': count.get('n',0)=0, count={'b':1,'a':1,'n':1}
char='a': count.get('a',0)=1, count={'b':1,'a':2,'n':1}
char='n': count.get('n',0)=1, count={'b':1,'a':2,'n':2}
char='a': count.get('a',0)=2, count={'b':1,'a':3,'n':2}
```

3.3 First Non-Repeating Character

```
# Input: "aabbcdd"  Output: "c" (first char appearing once)

def first_unique(s):
    count = {}
    for char in s:                    # Pass 1: count all
        count[char] = count.get(char, 0) + 1

    for char in s:                    # Pass 2: find first unique
        if count[char] == 1:
            return char

    return ""  # no unique character

# WHY two passes?
# Pass 1: gather ALL information first
# Pass 2: use the information to answer
# This "two-pass" approach is VERY common in interviews
```

```
"aabbcdd"
Pass 1: count = {a:2, b:2, c:1, d:2}
Pass 2: a->2 skip, a->2 skip, b->2 skip, b->2 skip,
        c->1 FOUND! Return "c"
```

3.4 Two Sum with Hash Map

The most famous interview problem. O(n) using complement lookup.

```
def two_sum(nums, target):
    seen = {}   # value -> index

    for i, num in enumerate(nums):
        complement = target - num    # what do I need?

        if complement in seen:       # found it!
            return [seen[complement], i]

        seen[num] = i                # remember this number

    return []
```

```
nums = [3, 5, 1, 8], target = 9

i=0, num=3: comp=9-3=6. 6 in {}? NO.      seen={3:0}
i=1, num=5: comp=9-5=4. 4 in {3:0}? NO.   seen={3:0, 5:1}
i=2, num=1: comp=9-1=8. 8 in {3:0,5:1}? NO. seen={3:0,5:1,1:2}
i=3, num=8: comp=9-8=1. 1 in {3:0,5:1,1:2}? YES!
  Return [seen[1], 3] = [2, 3]
  Check: nums[2]+nums[3] = 1+8 = 9  Correct!
```

3.5 Anagram Check

```
# Two strings are anagrams if same letters, different order
# "listen" <-> "silent"  True
# "hello" <-> "world"   False

def is_anagram(s1, s2):
    if len(s1) != len(s2):     # quick reject
        return False

    count = {}
    for char in s1:            # count s1 characters
        count[char] = count.get(char, 0) + 1

    for char in s2:            # subtract s2 characters
        count[char] = count.get(char, 0) - 1

    # If all counts are 0, it's an anagram
    return all(v == 0 for v in count.values())

# SHORTCUT:
def is_anagram(s1, s2):
    return sorted(s1) == sorted(s2)  # sort and compare

# Using Counter:
from collections import Counter
def is_anagram(s1, s2):
    return Counter(s1) == Counter(s2)
```

3.6 Group Anagrams (Medium)

```
# Input:  ["eat","tea","tan","ate","nat","bat"]
# Output: [["eat","tea","ate"], ["tan","nat"], ["bat"]]

# KEY INSIGHT: Sort each word's letters -> anagrams become same key
#   "eat" sorted -> "aet"
#   "tea" sorted -> "aet"  <- same!
#   "ate" sorted -> "aet"  <- same!

def group_anagrams(words):
    groups = {}
    for word in words:
        key = "".join(sorted(word))   # sort letters
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())

# Using defaultdict:
from collections import defaultdict
def group_anagrams(words):
    groups = defaultdict(list)
    for word in words:
        groups["".join(sorted(word))].append(word)
    return list(groups.values())
```

3.7 Majority Element

```
# Find element appearing more than n/2 times
# Input: [3, 3, 4, 2, 3, 3, 3]  Output: 3

def majority_element(nums):
    count = {}
    threshold = len(nums) / 2
    for num in nums:
        count[num] = count.get(num, 0) + 1
        if count[num] > threshold:
            return num     # return early!

# Boyer-Moore Voting (O(1) space - impressive to know)
def majority_element_optimal(nums):
    candidate = nums[0]
    count = 1
    for i in range(1, len(nums)):
        if count == 0:
            candidate = nums[i]
            count = 1
        elif nums[i] == candidate:
            count += 1
        else:
            count -= 1
    return candidate
```

3.8 Subarray Sum Equals K (Uses Prefix Sum + Hash Map)

```
# Count subarrays whose sum equals k
# This combines prefix sum with hash map!

def subarray_sum(nums, k):
    count = 0
    prefix_sum = 0
    seen = {0: 1}    # prefix_sum -> how many times seen

    for num in nums:
        prefix_sum += num

        # If (prefix_sum - k) was seen before,
        # there's a subarray summing to k
        if prefix_sum - k in seen:
            count += seen[prefix_sum - k]

        seen[prefix_sum] = seen.get(prefix_sum, 0) + 1

    return count
```

```
nums = [1, 2, 3], k = 3

num=1: prefix=1. 1-3=-2 in {0:1}? NO. seen={0:1,1:1}
num=2: prefix=3. 3-3=0 in {0:1}? YES! count=1. seen={0:1,1:1,3:1}
num=3: prefix=6. 6-3=3 in {0:1,1:1,3:1}? YES! count=2.

Answer: 2 (subarrays [1,2] and [3])
```

Hash Map Cheat Sheet

If You See...	Hash Map Stores...	Pattern Name
Count frequency	element -> count	Counting
Find duplicate	element -> bool/set	Existence check
Two sum / find pair	value -> index	Complement lookup
Group similar things	key -> list of items	Grouping
First/last occurrence	element -> index	Index mapping
Subarray sum = k	prefix_sum -> count	Prefix sum + map